

Learning Association-Line Transformations over Autosegmental Graphs

1. Introduction

This paper presents a method for learning transformations over multi-tier representations, with a particular focus on modifications of cross-tier relations (i.e., associations between tiers). The multi-tiered structures are widely attested in linguistics and are known as *Autosegmental Representations* (Clements and Goldsmith, 1984a; Goldsmith, 1976; Hayes, 1980; Sagey, 1986; Jardine, 2017), which are graph structures of temporally-ordered tiers and cross-tier association lines. This work focuses on a specific but important type of transformation where the elements on each tier remain unchanged from input to output, while the association lines are modified. We formalize these transformations as graph model structures (Strother-Garcia, 2019; Jardine and Heinz, 2015) and use a bottom-up factor inference method (Chandlee et al., 2019; Rawski, 2021) to infer which association lines are preserved, deleted, or newly introduced from observed input-output pairs.

Such multi-tiered structures have been widely used in linguistic theory, most notably in autosegmental phonology, to represent relations such as tone–syllable associations (Goldsmith, 1976), prosodic structure (Hayes, 1980), and feature geometry (Sagey, 1986). The core idea behind non-linear theory is that linguistic information can be organized into multiple tiers. Elements on different tiers are synchronized via association lines. The input-output mappings are in fact more naturally characterized by rearranging those association lines across tiers, rather than randomly changing features or properties of the elements themselves (Goldsmith, 1976; Clements and Goldsmith, 1984b; Odden, 2013; Hyman, 2009).

An example from Estako reduplication is shown in Figure 1, where segmental and tonal information is represented on separate tiers and connected by association lines. When reduplication creates a sequence of adjacent vowels /ówà/ + /ówà/ → /ówǎówà/, the final vowel of the first stem ǎ is deleted. Its associated L tone is then delinked from the deleted vowel and reassociated to the following vowel ó. As a result, the output is ówǎwà.

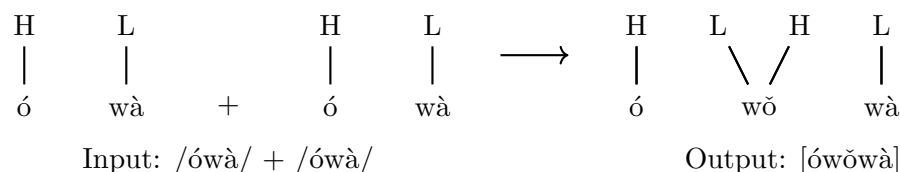


Figure 1: Association transformation in /ówà/ ‘house’ + /ówà/ ‘house’ → [ówǎwà] ‘every house’. Deletion of the first stem-final vowel triggers association change.

A large body of theoretical work on African tonal processes has demonstrated the advantages of autosegmental graphs (Clements and Goldsmith, 1984a; Odden, 2013; Hyman,

2009). Computationally, using multi-tiered structures has been shown to reduce the complexity of tonal patterns (Jardine, 2016; Chandlee and Jardine, 2021). Previous work has modeled these structures using finite-state approaches (Wiebe, 1992; Bird and Ellison, 1994; Yli-Jyrä, 2013, 2015; Rawski and Dolatian, 2020). These approaches typically convert structured representations into strings or synchronized tapes to leverage finite-state machines, or rely on hardcoding different types of associations into strings.

In contrast, we model tonal processes directly as transformations over autosegmental graphs. As a preliminary study, we focus on processes involving a single deassociation and/or reassociation operation, while the segmental content can be viewed unchanged between the underlying representation (UR) and the surface representation (SR).

We adopt the logical-transduction framework of Yolyan (2025a), which assumes a one-to-one correspondence between elements in the input and output structures. Each output element is interpreted as the surface realization of a corresponding input element, and phonological mappings are expressed by defining output association relations as functions of the input structure. This logical characterization also provides a model-theoretic framework for learning (Strother-Garcia, 2019) [is this accurate???].

This learning paradigm has proven effective in phonotactic learning from positive data using the Bottom-Up Factor Inference Algorithm (BUFIA; Chandlee et al. 2019; Rawski 2021; Li 2025; Li and Heinz to appear). BUFIA searches a partially ordered space of model-theoretic structures to identify a minimal hypothesis consistent with the data. For transformation learning, Yolyan (2025a,b) show that the target structure should distinguish positive from negative examples: it must be present in all positive examples and absent from all negative examples. Such a structure defines a minimal structural environment in which the transformation applies.

This work brings together these representational and learning perspectives. Using a case study of contour formation (as shown in Figure 1), we show how logical transductions over autosegmental graphs can be learned using a BUFIA-style learner. In contrast to previous work on string-based representations, the target of learning is a transformation of association relations between tiers. Although our demonstration is drawn from tonal phonology, the same approach can in principle be extended to other model-theoretically defined multi-tiered representations. To our knowledge, the learnability of association-line transformations has not previously been investigated.

2. Preliminaries

2.1. Autosegmental graphs

ASGs represent the idea that different aspects of linguistic structure are organized on distinct tiers while progressing simultaneously over time. Each tier encodes a particular type of information, such as tone (pitch information), segmental (tongue shape), syllabic (metrical) structure. Note that there may be multiple tiers in ASGs, provided that the elements on different tiers are disjoint. In most phonological applications, ASGs typically involve two or three tiers. In this work, we assume a two-tiered structure, though the definition remains the same in the general case. In addition, ASGs are subject to the No Crossing Constraint (NCC), which enforces temporal alignment between associations.

Definition 1 (Autosegmental Graph) An autosegmental graph (ASG) is a labeled $\mathcal{T}$ -partite graph whose vertices are partitioned into $\mathcal{T}$ disjoint sets, with each tier linearly-ordered. Equivalently, it can be defined model-theoretically as a relational structure $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$ where D is a finite domain of elements; $\mathcal{T} = \{T_1, \dots, T_n\}$ is a collection of tiers such that $T_1 \cup T_2 \cup \dots \cup T_n = D$ and $T_i \cap T_j = \emptyset$ for all $i \neq j$. $\triangleleft$ is a binary successor relation defined between nodes within tiers; $\alpha \subseteq D \times D$ is a set of non-crossing association relations across tiers.

Definition 2 (No Crossing Constraint (NCC)) An ASG needs to obey No Crossing Constraint. Formally, For all $(x, y), (x', y') \in \alpha$, $x \triangleleft x' \Rightarrow (y \triangleleft y') \vee (y = y')$. and vice versa.

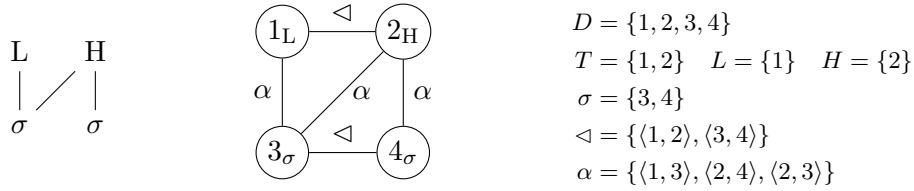


Figure 2: Three representations of a two-tier autosegmental graph.

Figure 2 shows a tone-syllable structure and its corresponding model. In this example, the first syllable is associated with both a Low (L) tone and a following High (H) tone, while the second syllable is associated with the H tone (often represented as LH.H in linguistic literature). The model consists of four labeled nodes, numbered 1, 2, 3, 4, each representing a linguistic unit with its corresponding property. There are two tiers: the tone tier T , containing the L and H tones, and the syllable tier σ , containing the two syllables. The successor relation, denoted by $\triangleleft$, represents the linear order within tiers. In this example, the successor relation is $\triangleleft = \{\langle 1, 2 \rangle, \langle 3, 4 \rangle\}$. The association relation, denoted by α , represents cross-tier associations: $\alpha = \{\langle 1, 3 \rangle, \langle 2, 3 \rangle, \langle 2, 4 \rangle\}$. Together, the tuple $\langle D, T, \sigma, L, H, \triangleleft, \alpha \rangle$ constitutes the *signature* of this ASG model.

Definition 3 (Size of ASG) Given $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$, the size of $\mathcal{A}$, denoted by $|\mathcal{A}|$, is defined as $|\mathcal{A}| = |D| + |\alpha|$.

The size of an autosegmental graph is defined as the sum of both node and association counts, excluding successor relations, since successor count is implicitly determined by the count of nodes on each tier. Note that node labels are not counted toward structure size, but instead serve only as refinements of already existing variables. For example, the simplest association structure connecting two nodes has size 3. The size of the ASG in Figure 2 is 7.

Lastly, for convenience, we also define the notion of neighbors in an ASG. Two nodes are considered neighbors if they are connected by a successor relation.

Definition 4 (Neighbor) Given $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$, a node n is a neighbor of a node m iff $m \triangleleft n \vee n \triangleleft m$.

2.2. Matrix Representation

In addition to representing model-theoretic relational structures with graphs, one can use *adjacency matrices* which use binary values to denote whether there is an edge between two nodes. Two nodes are considered *adjacent* if there is an edge connecting them. For example, the ASG shown in Figure 2 can be equivalently represented as in Table 1.

Definition 5 (Adjacency matrices) *Given an ASG $\mathcal{A} = \langle D, \mathcal{T}, \alpha, \triangleleft \rangle$ with t tiers, the association relation α can be represented by $t - 1$ adjacency matrices, each corresponding to the association relation between two adjacent tiers $T_i, T_j \in \mathcal{T}$. The adjacency matrix $M_\alpha^{(i,j)}$ is defined such that*

$$M_\alpha^{(i,j)}[k, \ell] = \begin{cases} 1 & \text{if } (x_k, y_\ell) \in \alpha, \\ 0 & \text{otherwise,} \end{cases}$$

where $x_k \in T_i$ and $y_\ell \in T_j$.

	L	H
σ_1	1	1
σ_2	0	1

Table 1: Adjacency matrix representation of the ASG in Figure 2.

The motivation for adopting matrices is to establish a framework corresponding to string-based learning. Concepts such as substrings, subfactors, and superfactors are well developed and widely used in grammar inference (Vu et al., 2018; Lambert et al., 2021), but have not been deeply explored in terms of ASGs. To make these concepts applicable to ASGs, we follow the notions of *restriction* and *containment* proposed in Chandlee et al. (2019), adapting them to graph-based representations through the notions of *submatrices* and *matrix containment*. (Li and Heinz, to appear)

Definition 6 (Submatrix) *Let matrix A be an $m \times n$ matrix. A matrix B is a submatrix of A if it is formed by taking k ($1 \leq k \leq m$) contiguous rows and v ($1 \leq v \leq n$) contiguous columns from A ¹.*

Definition 7 (Containment) *An $m \times n$ matrix $A = [a_{ij}]$ is contained in a matrix B ($A \sqsubseteq B$) if there exists an $m \times n$ submatrix of B , denoted $B' = [b'_{ij}]$, such that for all $i \in [1, m]$ and $j \in [1, n]$, $b'_{ij} \geq a_{ij}$.*

A matrix A is a *subfactor* of B if and only if A is contained in B . Conversely, B is a *superfactor* of A . Furthermore, if the size difference between a superfactor and its subfactor is exactly one, the superfactor is called the *next superfactor* of that subfactor.

This containment relation is important because it provides a *partial ordering*, which enables the learner to infer entailment relations among structures. We illustrate this partial

1. The term “submatrix” is generally defined by deleting arbitrary rows and/or columns from the original matrix. However, in our case, we require the remaining rows and columns must be *contiguous* in the original matrix, which is related to the notion of *connectedness* in Chandlee et al. (2019).

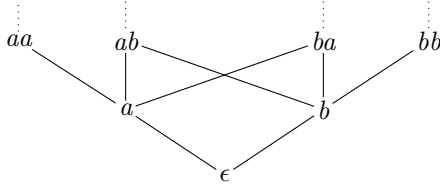


Figure 3: Strings over $\Sigma = \{a, b\}$ ordered by the containment relation (non-exhaustive)

order using the toy string examples in Figure 3. A superfactor is connected to its subfactors, which are positioned below it in the diagram and are contained within it. This structure facilitates the learner to infer entailment relations among factors. In particular, if a factor is ungrammatical, all of its superfactors are likewise ungrammatical; conversely, if a factor is grammatical, all of its subfactors are grammatical as well. Readers are referred to other related work (Chandlee et al., 2019; Rawski, 2021; Swanson, 2025; Swanson et al., 2026; Payne, 2024; Li, 2025) for a more detailed discussion of the partial ordering itself. Here, we emphasize that this partial order forms the foundation of bottom-up learning algorithms, since it provides a principled way to navigate the hypothesis space through entailment relations between factors.

3. Logical Transduction

Using model theory (Strother-Garcia, 2019; Yolyan, 2025a; Jardine, 2017), we model a phonological process as a relation between input and output structures $\mathcal{A}_i \rightarrow \mathcal{A}_o$. There have been work adopting the same fashion on string-to-string transduction [NEED REF], but the essentially the logic is not restricted to strings and can operate over any model-theoretic representations. In this framework, we use a set of logic formula to derive the output signature with reference to the input signature. Recall from the previous section that we defined an ASG signature as $\mathcal{A} = \langle \mathcal{T}, \alpha, \triangleleft \rangle$. Given an input and output models with this signature, the output signature can be defined in terms of the input structure.

Output	Input	Interpretation
ϕ_{Domain}	$:= \text{TRUE}$	Preserve all input elements
COPY	$:= 1$	Define Output domain the same size as Input
$\phi_{\text{license}}(x)$	$:= \exists y \alpha(x, y)$	Preserve only associated elements
ϕ_T	$:= T(x)$	Preserve elements and their labels
$\phi_{\triangleleft'(x, y)}$	$\approx \triangleleft(x, y)$	Preserve linear order within tiers
$\phi_{\alpha'}(x, y)$	$:= (\alpha(x, y) \wedge \neg \text{deassoc}(x, y)) \vee (\neg \alpha(x, y) \wedge \text{reassoc}(x, y))$	Update association relations

Table 2

As shown in Table 2, the transduction defines an output model with the same domain as the input structure, while filtering out unassociated elements by $\phi_{\text{license}}(x)$. All unary predicates $T(x)$ in the input signature are preserved in the corresponding output predicates

$T'(x)$, meaning that an element x has property T in the output iff it has property T in the input. Similarly, $\triangleleft'(x, y) \approx \triangleleft(x, y)$ indicates the within-tier order relation is preserved. The only change is the association relation α : an association line holds in the output iff it is (i) present in the input and not removed by **deassoc**, or (ii) absent in the input but newly introduced by **reassoc**.

$$\phi_{\alpha'(x,y)} = (\alpha(x,y) \wedge \neg \text{deassoc}(x,y)) \vee (\neg \alpha(x,y) \wedge \text{reassoc}(x,y))$$

In this way, the problem of learning association-line transformations can be reduced to learning the logical formulas for **deassoc** and/or **reassoc**. When no transformation occurs between the input and output representations, neither **deassoc** nor **reassoc** takes place (i.e., both have the value of false). The association relation therefore remains unchanged, and all input associations are faithfully preserved in the output. Since these two predicates are combined through disjunction, two parallel learning processes apply simultaneously, and their outputs are combined to define the final association relation α' .

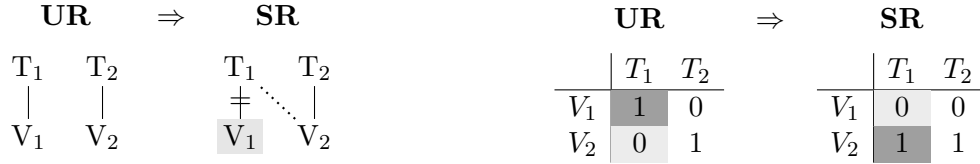


Figure 4: Contour formation in both graph-theoretic and matrix representations.

Consider a common phenomenon, contour formation, as shown in Figure 1. Figure 4 visualizes this process in both graph-theoretic and matrix representations. From a logical perspective, this transformation is captured by three operations: **deassoc**, **reassoc**, and ϕ_{license} . The **deassoc** function identifies the association to be broken between T_1 and the deleted vowel V_1 (the crossed-out line). The conjunction $\alpha(x, y) \wedge \neg \text{deassoc}(x, y)$ removes this association (T_1, V_1) while preserving all unaffected associations, that is, (T_2, V_2) . The **reassoc** function then links T_1 to the surviving vowel V_2 (the dotted line). As a result, V_1 is no longer associated with any element on the opposite tier and therefore fails to satisfy the licensing condition ϕ_{license} in the output. Consequently, V_1 is not licensed and is omitted from the surface form (shown in gray).

The **deassoc** and **reassoc** operations are equivalently reflected in the adjacency matrices in Figure 4. The value for (T_1, V_1) changes from 1 (in the input) to 0 (in the output), which indicates delinking; the value corresponding to (T_1, V_2) changes from 0 to 1, which indicates reassociation. Formally, **deassoc** and **reassoc** can also be identified by $\text{deassoc} = \{(i, j) \mid \text{UR}_{ij} = 1 \wedge \text{SR}_{ij} = 0\}$ and $\text{reassoc} = \{(i, j) \mid \text{UR}_{ij} = 0 \wedge \text{SR}_{ij} = 1\}$.

4. Association Learning

This section follows the framework of using logical transductions to learn phonological mappings in Yolyan (2025a), which leverages the entailment relation in a partially ordered space and updates hypotheses about the smallest contextual environments that trigger a featural change (a string-based input-output mapping that can be written as $A \rightarrow B / C_D$).

Several aspects of this framework are particularly important for the present work. First, phonological processes are represented as logical formulas over model-theoretic representations. Second, the learning procedure relies on grammatical entailment relations within a partially ordered hypothesis space, with more general structures at the bottom and more specific structures higher up (Chandlee et al., 2019; Rawski, 2021). If an environment (which itself is a structure, thereafter we call it environment) is sufficient to trigger a transformation, then any superfactor containing that environment is likewise sufficient to trigger the same transformation. On the other hand, if an environment fails to trigger the change, then all of its subfactors are likewise insufficient to trigger the change. In other words, given a hypothesized environment, all data satisfying this condition are expected to undergo the transformation (positive evidence), while all data not satisfying this condition are expected not to undergo the transformation (negative evidence). If some positive data fail to undergo the transformation, then the hypothesis is too restrictive. Conversely, if some negative data nevertheless undergo the transformation, then the hypothesis is too general. The goal of learning is therefore to identify the smallest common environment that are contained in all positive evidence, but absent in all negative evidence. For example, post-nasal voicing can be written as $-voi \rightarrow +voi / +nasal, -cont_$. This rule states that whenever a segment specified as $[-voi]$ occurs after a segment with features $[+nasal, -cont]$, its voicing feature changes to $[+voi]$.

In autosegmental phonology, however, contextual environments are not expressed linearly as $A \rightarrow B / C_D$, but rather as structural relations over graphs. Because the learning framework applies generally to model-theoretic representations, it extends naturally to graph-based representations. Equivalently, transformations can be viewed as changes to the graph’s adjacency matrix, where an entry changes from $1 \rightarrow 0$ or $1 \rightarrow 0$.

$$\begin{array}{ccc}
 & ? & ? & ? \\
 0 \rightarrow 1 / & ? & - & ? \\
 & ? & ? & ?
 \end{array}
 \quad \text{or} \quad
 \begin{array}{ccc}
 & ? & ? & ? \\
 1 \rightarrow 0 / & ? & - & ? \\
 & ? & ? & ?
 \end{array}$$

To learn ASGs, we adapt the bottom-up factor inference framework of Chandlee et al. (2019) and the logical learning framework of Yolyan (2025a) to graph-based autosegmental representations (Li and Heinz, to appear). Initially, the learner identifies changing association sites by comparing the input and output association matrices. If a deassociation is found, the learner then initializes the search with the corresponding most general logical hypothesis: $\alpha(x, y)$ if a reassociation is found, then the learner starts with $neg\alpha(x, y)$ ². The procedure is shown in Algorithm 1.

Change the algo

The input data consists of a finite set of input-output ASG pairs, which can be partitioned into positive and negative evidence, denoted V^+ and V^- respectively. It also requires a bound k that limits the graph size.

The algorithm conducts a breadth-first search over a partially ordered hypothesis space. At each step, a candidate hypothesis c is first tested against the positive evidence: if c is not contained in all positive examples, then the current hypothesis is incompatible with the

2. We treat deassociation and reassociation as two independent processes and combine the resulting hypotheses through disjunction.

Data: $D = \{(I_1, O_1), \dots, (I_n, O_n)\}, V^+ = \{I_n \mid I_n \neq O_n\}, V^- = \{I_n \mid I_n = O_n\}, k, type$

```

1  $P \leftarrow \emptyset$ 
2  $Q \leftarrow \{\alpha(x, y)\}$  or  $Q \leftarrow \{\neg\alpha(x, y)\}$ 
3 while  $Q \neq \emptyset$  do
4    $c \leftarrow Q.Dequeue()$ 
5   if  $\forall g \in V^+, c \sqsubseteq g$  then
6     if  $\forall g \in V^-, c \not\sqsubseteq g$  then
7       return  $c$ 
8     else
9       foreach  $s \in \text{NEXTSUPFACT}(c) \wedge |s| < k \wedge \forall p \in P, s \not\sqsubseteq p$  do
10         $Q.Enqueue(s)$ 
11   else
12      $P \leftarrow P \cup \{c\}$ 
13 return  $\perp$ 

```

Algorithm 1: Learning Association Changes

observed transformation and thus is added to the pruning set P . Otherwise, the learner checks c against all negative examples: if c is NOT contained in any datum in V^- , then the current environment is output as it is consistent with all data. Otherwise, the hypothesis is considered too general, and will be updated minimally using NEXTSUPFACT. Meanwhile, the learner filters out candidate environments that contain pruned structures or exceed the predetermined complexity bound k , so as to keep the search space tractable. Since hypotheses are explored breadth-first, the first valid hypothesis returned by the learner is guaranteed to be the minimal structural environment consistent with all positive and negative evidence.

In terms of NEXTSUPFACT, there are many possible ways to generate superfactors while still covering the same hypothesis space. Here, we propose one possible method that explicitly connects graph-theoretic representations with their logical counterparts.

Recall Definition 3 defines the size of an ASG as the total number of elements and association lines. and states node labels are not counted toward structure size. In addition, the next superfactors of a structure are those whose size increases minimally by 1. Consequently, increases in size can only arise through the introduction of new elements or new association relations.

Therefore, when generating next superfactors, the learner follows Occam's Razor by preferring the smallest possible structure and expanding the domain only when required by the data. Specifically, see the following order:

- (i) labeling unspecified nodes; if labeling does not contribute to the transformation, then backtracks to the under-specified variable.
- (ii) introducing a neighboring node, which may either follow or precede an existing node on either tier, at both boundaries;
- (iii) adding association lines, provided that both endpoints are already present in the structure.

Table 3 presents a non-exhaustive set of examples illustrating the generation of NEXTSUPFACT from an initial deassoc hypothesis.

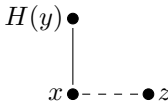
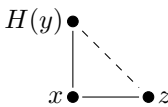
Operation	Matrix	Graph	Logic	Size
Initial	$\begin{array}{c c} & x \\ \hline y & 1 \end{array}$		$\alpha(x, y)$	3
Add label	$\begin{array}{c c} & H \\ \hline y & 1 \end{array}$		$\alpha(x, y) \wedge H(y)$	3
Add node	$\begin{array}{c cc} & H & z \\ \hline y & 1 & \end{array}$		$\alpha(x, y) \wedge H(x) \wedge \exists z \triangleleft (x, z)$	4
Add association	$\begin{array}{c cc} & H & z \\ \hline y & 1 & 1 \end{array}$		$\alpha(x, y) \wedge H(x) \wedge \exists z (\triangleleft (x, z) \wedge \alpha(y, z))$	5

Table 3: Examples of operations used in generating NEXTSUPFACT hypotheses across matrix, graph, and logical representations.

As can be seen, each operation provides an equivalent interpretation from matrix, graph-theoretic, and logical perspectives. New quantifiers are only introduced through successor relations, corresponding to the addition of neighboring nodes in the graph structure. Within the same quantifier scope, the learner may further specialize the hypothesis through conjunction, including adding association relations (corresponding to changing matrix cells) or specifying node labels and feature values.

5. Case Study: Contour Formation

We use the contour formation to demonstrate both the logical transduction framework and BUFIA-style learning over ASGs. We assume a model $\mathcal{M} = \{H, L, C, V, \alpha, \triangleleft\}$, where H and L are tonal elements on the tone tier T , while C and V belong to the segmental tier S . Table 4 gives five input-output pairs, and cells marked with a dash indicate the same output as the input. The headers provide shorthand notations for each pair. The contour formation process can be observed in the CVV-LH and CVV-HL examples, where both tones become associated with the same vowel. This transformation can equivalently be represented as a change in the association matrix. Notably, tonal identities themselves are irrelevant to the deassociation and reassociation process: both CVV-LH and CVV-HL undergo contour formation once the two tones are linked to a single vowel. By contrast, an LH sequence does not necessarily surface as a contour; whether it does depends on the structural configuration of the vowels. For example, the LH sequence surfaces as a contour in CVV-LH, where the vowels are adjacent, but not in VCV-LH, where they are separated.

The learner first partitions the data into positive evidence V^+ and negative evidence V^- . In this toy example, V^+ consists of CVV-LH and CVV-HL, and V^- consists of the rest pairs marked with dashes in Table 4. By comparing V^+ , the learner can thus observe both

	CV-H	CV-L	VCV-LH	CVV-LH	CVV-HL
UR	$\begin{array}{c} \text{H} \\ \\ \text{C} \quad \text{V} \end{array}$	$\begin{array}{c} \text{L} \\ \\ \text{C} \quad \text{V} \end{array}$	$\begin{array}{cc} \text{L} & \text{H} \\ & \\ \text{V} & \text{C} \quad \text{V} \end{array}$	$\begin{array}{cc} \text{L} & \text{H} \\ & \\ \text{C} & \text{V}_1 \quad \text{V}_2 \end{array}$	$\begin{array}{cc} \text{H} & \text{L} \\ & \\ \text{V}_1 & \text{V}_2 \end{array}$
SR	–	–	–	$\begin{array}{cc} \text{L} & \text{H} \\ & \diagdown \quad \\ \text{C} & \text{V}_2 \end{array}$	$\begin{array}{cc} \text{H} & \text{L} \\ & \diagdown \quad \\ & \text{V}_2 \end{array}$

Table 4: Toy ASG

$$A_i = \begin{array}{c|cc} & T & T \\ \hline C & 0 & 0 \\ V & 1 & 0 \\ V & 0 & 1 \end{array} \Rightarrow A_o = \begin{array}{c|cc} & T & T \\ \hline C & 0 & 0 \\ V & 0 & 0 \\ V & 1 & 1 \end{array}$$

Table 5: Input-output matrices (A_i) and (A_o) contour formation. The highlighted entries indicate changes in association relations.

deassociation and reassociation occur. Here, we will only demonstrate the deassociation learning since the reassociation proceeds analogously.

For deassociation, the learner then begins with the following initial hypothesis, which states that any association line present in the input is deleted in the output.

$$H_0 : \quad \phi_{\alpha'(x,y)} = \alpha(x, y)$$

The following step is to test H_0 against both positive and negative evidence. Trivially, it is accepted by all positive examples, but, when evaluated against the negative evidence, the hypothesis is rejected: in CV-H, CV-L, and VCV-LH, the association lines in the input remain associated in the output, contrary to the prediction of H_0 . The learner therefore concludes that the hypothesis is too general and must be made more specific.

To refine the hypothesis, the learner generates increasingly restricted structures by introducing additional structural conditions over the ASG. Following a label-node-association order, the learner first constructs next superfactors by specializing the hypothesis through node labeling. Naively, the learner iterates over all symbols available on two corresponding tiers and substitutes them into the existing unlabeled nodes. The resulting factors are shown in Table 6: (6a) and (6b) label y as H or L on the tone tier, while (6c) and (6d) label x as V or C on the segment tier.

When evaluating the candidate hypotheses in Table 6, we find that neither (6a) nor (6b) covers all positive examples. This indicates that tonal identity does not contribute to the transformation, and the learner therefore backtracks to the underspecified node. Likewise, on the segmental tier, (6d) is inconsistent with the positive evidence and is rejected, whereas (6c) remains consistent with all positive examples. The learner therefore refines the hypothesis as follows:

$$H_1 : \quad \phi_{\alpha'(x,y)} = \alpha(x, y) \wedge V(x)$$

	(6a)	(6b)	(6c)	(6d)
Label Nodes	$H(y) \bullet$ $\downarrow$ $x \bullet$	$L(y) \bullet$ $\downarrow$ $x \bullet$	$y \bullet$ $\downarrow$ $V(x) \bullet$	$y \bullet$ $\downarrow$ $C(x) \bullet$
Contained by all V^+	$\times$	$\times$	$\checkmark$	$\times$
Absent in V^-	$\times$	$\times$	$\times$	$\times$

 Table 6: Next Superfactors of H_0

However, since (6c) is also attested in negative examples, it fails to distinguish contexts in which the transformation applies from those in which it does not. The learner therefore generates the next superfactors of (6c) by introducing adjacent nodes on either tier. As the previous step established that tonal identity is not relevant to the transformation, node y remains underspecified rather than being assigned a tonal label. The resulting factors are shown in Table 7, where (7a)–(7b) introduce a new node that immediately precedes or follows the existing node on the segment tier, while (7c)–(7d) do so on the tone tier.

	(7a)	(7b)	(7c)	(7d)
	$y \bullet$ $\downarrow$ $V(x) \bullet \text{---} \bullet z$	$y \bullet$ $\downarrow$ $z \bullet \text{---} \bullet V(x)$	$z \bullet \text{---} \bullet y$ $\downarrow$ $\bullet V(x)$	$y \bullet \text{---} \bullet z$ $\downarrow$ $V(x) \bullet$
Contained by all V^+	$\checkmark$	$\times$	$\times$	$\checkmark$
Absent in all V^-	$\times$	$\times$	$\times$	$\times$

 Table 7: Next Superfactors of (6c) in H_1

Again, the following step is to test against V^+ and V^- . In this round, two candidates, namely (7a) and (7d), are contained by all positive evidence. At the same time, the learner rejects (7b) and (7c) and prunes them from hypothesis space. This suggests that the current deassociation process is insensitive to elements on the left side of the deletion site, whether on the tonal tier or the segmental tier, but depends on rightward context. Pruning off (7b) and (7c) eliminates all subsequent hypotheses that introduce leftward contextual conditions but only restricts the search space to rightward environments only.

Based on candidates (7a) and (7d), the learner has the following updated hypotheses:

$$H_{2a} : \quad \phi_{\alpha'(x,y)} = \alpha(x, y) \wedge V(x) \wedge \exists z(x \triangleleft z)$$

$$H_{2d} : \quad \phi_{\alpha'(x,y)} = \alpha(x, y) \wedge V(x) \wedge \exists z(y \triangleleft z)$$

However, both remaining candidates are still present in some negative examples and therefore remain too general and need further refinement. At this stage, the learner could introduce additional nodes or add associations between z and a node on the opposite tier, but either operation would increase the size of the hypothesis. Labeling, by contrast, does

not increase complexity. Consequently, the learner specializes the newly introduced node z . The resulting factors are shown in Table 8. Factors (8a)–(8b) refine H_{2a} by labeling z as V or C on the segmental tier, whereas (8c)–(8d) label z as H or L on the tonal tier.

	(8a)	(8b)	(8c)	(8d)
	$\begin{array}{c} \bullet y \\ \\ V(x) \bullet \text{---} \bullet V(z) \end{array}$	$\begin{array}{c} \bullet y \\ \\ V(x) \bullet \text{---} \bullet C(z) \end{array}$	$\begin{array}{c} y \bullet \text{---} \bullet H(z) \\ \\ \bullet V(x) \end{array}$	$\begin{array}{c} y \bullet \text{---} \bullet L(z) \\ \\ \bullet V(x) \end{array}$
Contained by all V^+	✓	×	×	×
Absent in V^-	✓	×	×	×

Table 8: Next Superfactor of H_{2a} and H_{2d}

Only candidate (8a) is both contained in all positive evidence and absent from all negative evidence. The learner therefore converges on this hypothesis as a minimal structure that triggers the transformation and is consistent with both positive and negative evidence. This formula encodes the same structural condition illustrated in Figure 4.

$$\phi_{\alpha'(x,y)} = \alpha(x, y) \wedge V(x) \wedge \exists z((x \triangleleft z) \wedge V(z))$$

Importantly, one may observe that triggering deassociation does not require T_2 to be associated with V_2 . The learned hypothesis therefore predicts that deassociation applies whenever two vowels are adjacent, regardless of whether the following vowel is associated with a tone. **[delete or stay??]**

The reassoc inference follows the same process but with a different starting hypothesis

$$\phi_{\alpha'(x,y)}[\text{we need to have two diff predicate names}] = \neg\alpha(x, y)$$

In this process, the triggering context lies to the left of the reassociation site, in contrast to the deassociation case. The learner eventually converges on the following hypothesis:

$$\phi_{\alpha'(x,y)} = \neg\alpha(x, y) \wedge V(x) \wedge \exists z((z \triangleleft x) \wedge V(z) \wedge \alpha(z, y))$$

We combine the two operations into the final transformation.

$$\begin{aligned} \phi_{\alpha'(x,y)} &:= \alpha(x, y) \wedge V(x) \wedge \exists z((x \triangleleft z) \wedge V(z)) \\ &\vee \neg\alpha(x, y) \wedge V(x) \wedge \exists z((z \triangleleft x) \wedge V(z) \wedge \alpha(z, y)) \end{aligned}$$

Lastly, the licensing function ϕ_{license} filters out elements that are not associated in the surface representation. Such unlicensed elements are not realized phonetically and therefore do not appear in the output.

6. Conclusion

This work has argued that phonological processes that involve multi-tiered structures such as autosegmental representations can be uniformly captured using model theory and logical

transductions. The input-output mapping can be modeled by modifying a single relation, α , through two operations of **deassoc** and **reassoc**. We also show that BUFIA-style learning can be naturally extended to such enriched representations by traversing a partially ordered hypothesis space. A crucial component is NEXTSUPFACTOR, which generates minimally more complex structures through node labeling, the introduction of neighboring nodes, and the addition of association relations, to keep the search space tractable. The resulting hypothesis is a structural environment, or equivalently a logical formula, that is consistent with both the positive and negative evidence.

An important direction for future work is to relax the current restriction to single deassociation and reassociation processes. Doing so will require learning more complex hypotheses involving conjunctions of structural conditions. Another limitation of the current analysis is that the transduction relies on quantification to capture across-tier dependencies. Future work can determine whether these processes can be simplified in a quantifier-free (QF) process. This opens up more discussions about how linguistically meaningful representations can facilitate grammar inference.

References

- Steven Bird and T Mark Ellison. One-level phonology: Autosegmental representations and rules as finite automata. *Computational Linguistics*, 20(1):55–90, 1994.
- Jane Chandlee and Adam Jardine. Input and output locality and representation. *Glossa: a journal of general linguistics*, 6(1), 2021.
- Jane Chandlee, Rémi Eyraud, Jeffrey Heinz, Adam Jardine, and Jonathan Rawski. Learning with partially ordered representations. In *Proceedings of MOL16*, pages 91–101, 2019.
- George N Clements and J Goldsmith. Autosegmental studies in bantu tone, 1984a.
- George N Clements and John Goldsmith. Autosegmental studies in bantu tone: Introduction. *Clements & Goldsmith (1984b)*, 117, 1984b.
- John Anton Goldsmith. *Autosegmental phonology*. PhD thesis, Massachusetts Institute of Technology, 1976.
- Bruce Philip Hayes. *A metrical theory of stress rules*. PhD thesis, Massachusetts Institute of Technology, 1980.
- Larry M Hyman. The representation of tone. *UC Berkeley PhonLab Annual Report*, 5(5), 2009.
- Adam Jardine. Computationally, tone is different. *Phonology*, 33(2):247–283, 2016.
- Adam Jardine. The local nature of tone-association patterns. *Phonology*, 34(2):363–384, 2017.
- Adam Jardine and Jeffrey Heinz. A concatenation operation to derive autosegmental graphs. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 2015)*, pages 139–151, 2015.

- Dakotah Lambert, Jonathan Rawski, and Jeffrey Heinz. Typology emerges from simplicity in representations and learning. *Journal of Language Modelling*, 9, 2021.
- Han Li. Learning tonotactic patterns over autosegmental representations. In *Proceedings of the Annual Meetings on Phonology*, volume 1. University of Massachusetts Amherst Libraries, 2025.
- Han Li and Jeffrey Heinz. Learning tonotactics with autosegmental representations. *Phonology*, to appear.
- David Odden. *Introducing Phonology*. Cambridge University Press, Cambridge, 2 edition, 2013.
- Sarah Payne. A generalized algorithm for learning positive and negative grammars with unconventional string models. In *Proceedings of the Society for Computation in Linguistics 2024*, pages 75–85, 2024.
- Jonathan Rawski. *Structure and learning in natural language*. PhD thesis, State University of New York at Stony Brook, 2021.
- Jonathan Rawski and Hossep Dolatian. Multi-input strictly local functions for tonal phonology. In *Proceedings of the Society for Computation in Linguistics 2020*, pages 208–223, 2020.
- Elizabeth Caroline Sagey. *The representation of features and relations in non-linear phonology*. PhD thesis, Massachusetts Institute of Technology, 1986.
- Kristina Strother-Garcia. *Using model theory in phonology: a novel characterization of syllable structure and syllabification*. University of Delaware, 2019.
- Logan Swanson. Learning multi tier-based strictly 2-local languages. In *Proceedings of the 18th Meeting on the Mathematics of Language*, pages 33–43, 2025.
- Logan Swanson, Jeffrey Heinz, and Jonathan Rawski. Phonotactic learning with structure, not statistics. *Linguistic Inquiry*, pages 1–24, 05 2026.
- Mai H. Vu, Ashkan Zehfroosh, Kristina Strother-Garcia, Michael Sebok, Jeffrey Heinz, and Herbert G. Tanner. Statistical relational learning with unconventional string models. *Frontiers in Robotics and AI*, Volume 5 - 2018, 2018. ISSN 2296-9144. doi: 10.3389/frobt.2018.00076.
- Bruce Wiebe. Modelling autosegmental phonology with multi-tape finite state transducers. 1992.
- Anssi Yli-Jyrä. On finite-state tonology with autosegmental representations. In *Proceedings of the 11th international conference on finite state methods and natural language processing*, pages 90–98, 2013.
- Anssi Yli-Jyrä. Three equivalent codes for autosegmental representations. In *Proceedings of the 12th International Conference on Finite-State Methods and Natural Language Processing 2015 (FSMNLP 2015 Düsseldorf)*, 2015.

Tatevik Yolyan. A framework for learning phonological maps as logical transductions. In *Proceedings of MOL18*, pages 44–58, 2025a.

Tatevik Arayevna Yolyan. *Phonological Expressivity and Learning via Boolean Monadic Recursive Schemes*. PhD thesis, Rutgers The State University of New Jersey, School of Graduate Studies, 2025b.